A MINI PROJECT REPORT ON

DIABETES PREDICTION USING MACHINE LEARNING

Submitted in Partial Fulfilment of the Requirements for the

Artificial Intelligence Laboratory

Course Code: CS591

for the Degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

SUBMITTED BY

Name	Univ. Roll No.
KUNAL SINGH	108001240085
SAIKAT SINHA	108001240126
RAKESH PAL	108001240115
SAYAN MONDAL	108001240133
RISHAVRAJ	108001240117

UNDER THE GUIDANCE

OF

Mr. Pranabes Gangopadhyay

<Designation>

Department of Computer Science and Engineering



DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

ASANSOL ENGINEERING COLLEGE

An Autonomous Institute

Affiliated to

MAULANA ABUL KALAM AZAD UNIVERSITY OF TECHNOLOGY

November, 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGNEERING
ASANSOL ENGINEERING COLLEGE
Vivekananda Sarani, Kanyapur, Asansol, West Bengal – 713305

RECOMMENDATION

I hereby recommend that the project titled "**Diabetes Prediction Using Machine Learning**", submitted by the following students, be accepted in partial fulfilment of the requirements for the Artificial Intelligence Laboratory (Course Code: CS591) of the B.Tech. programme in Computer Science and Engineering at Asansol Engineering College, an autonomous institute affiliated with Maulana Abul Kalam Azad University of Technology, West Bengal.

Project Group Members

Sl. No	University Roll No.	Student's Name
1	108001240085	KUNAL SINGH
2	108001240126	SAIKAT SINHA
3	108001240115	RAKESH PAL
4	108001240133	SAYAN MONDAL
5	108001240117	RISHAV RAJ

Place: Asansol

Date: _____

Project Guide's Signature: _____

Mr. Pranabes Gangopadhyay
<Designation>
Department of Computer Science and Engineering
Asansol Engineering College

DEPARTMENT OF COMPUTER SCIENCE AND ENGNEERING
ASANSOL ENGINEERING COLLEGE
Vivekananda Sarani, Kanyapur, Asansol, West Bengal – 713305

CERTIFICATE OF APPROVAL

This is to certify that the project entitled "**Diabetes Prediction Using Machine Learning**", submitted by the students of the Department of Computer Science and Engineering, Asansol Engineering College, has been carried out as part of the requirements of the Artificial Intelligence Laboratory (Course Code: CS591) during the 5th Semester of the B.Tech. programme in Computer Science and Engineering.

The project work has been carried out under the guidance and supervision of the undersigned. This report describes the supplied diabetes dataset, the implemented preprocessing and model-comparison workflow, and a Streamlit demonstration based on the saved Random Forest classifier.

The report is approved as a satisfactory academic study submitted in partial fulfilment of the requirements of the Artificial Intelligence Laboratory course.

This approval does not necessarily imply endorsement of the opinions or conclusions expressed in the report.

Place: Asansol

Date: _____

Signature of Project Guide	Signature of Head of Department
Mr. Pranabes Gangopadhyay	**Dr. Avishek Banerjee**
<Designation>	Department of Computer Science and Engineering
Department of Computer Science and Engineering	Asansol Engineering College
Asansol Engineering College	

DEPARTMENT OF COMPUTER SCIENCE AND ENGNEERING
ASANSOL ENGINEERING COLLEGE
Vivekananda Sarani, Kanyapur, Asansol, West Bengal – 713305

ACKNOWLEDGEMENT

We, the undersigned students, declare that this report presents our project work and that the sources used are acknowledged in the references. We also express our sincere gratitude to our project supervisor, Mr. Pranabes Gangopadhyay (PG sir), <Designation>, Department of Computer Science and Engineering, Asansol Engineering College, for guidance and support throughout this project.

We are thankful to the Department of Computer Science and Engineering, Asansol Engineering College, for providing the necessary facilities and academic environment to carry out this project as part of the Artificial Intelligence Laboratory (Course Code: CS591).

Finally, we thank everyone who directly or indirectly contributed to the successful completion of this project.

Project Group Members

Roll No	Student's Name	Signature
108001240085	KUNAL SINGH	
108001240126	SAIKAT SINHA	
108001240115	RAKESH PAL	
108001240133	SAYAN MONDAL	
108001240117	RISHAV RAJ	

ABSTRACT

This project develops an educational machine-learning workflow for classifying diabetes outcome from eight health-related attributes. The supplied CSV contains 768 records, eight predictors, and a binary target. The notebook treats zero values in glucose, blood pressure, skin thickness, insulin, and BMI as coded missing measurements and replaces them with column means. A stratified 80:20 split with random_state 42 produces 614 training and 154 test records. Six classifiers are compared using accuracy, precision, recall, F1-score, and ROC-AUC. In the recorded experiment, Random Forest gives the highest ROC-AUC (0.8194) and accuracy (74.68%); KNN has the highest positive-class recall (0.6296) and F1-score (0.6239). A Streamlit interface accepts eight values and displays a Random Forest class and score. The notebook imputes before splitting, and the app does not repeat that preprocessing, so results need a consistent pipeline and stronger validation. The prototype is for coursework and is not a medical diagnostic tool.

Keywords: diabetes classification, Random Forest, Pima-style dataset, model evaluation, Streamlit, mean imputation

TABLE OF CONTENTS

Section	Page
Recommendation	2
Certificate of Approval	3
Candidate Declaration	4
Acknowledgements	5
Abstract	6
List of Figures	8
List of Tables	9
CHAPTER 1: INTRODUCTION	**10**
1.1 Introduction and Problem Statement	10
1.2 Objectives of the Project	10
1.3 Scope of the Project	11
CHAPTER 2: LITERATURE SURVEY	**12**
2.1 Review of Existing Work	12
2.2 Summary of Literature Survey	13
2.3 Research Gap	13
CHAPTER 3: DATASET AND PROPOSED METHODOLOGY	**14**
3.1 Dataset Description / Input Data	14
3.2 Data Preprocessing / Input Preparation	14
3.3 Algorithm / Model / Core Technique	15
3.4 Development, Testing and Validation	16
3.5 Proposed Solution and System Workflow	16
CHAPTER 4: IMPLEMENTATION AND RESULTS	**18**
4.1 Development Environment and Tools	18
4.2 Implementation	18
4.3 Experimental Results	20
4.4 Performance Evaluation Metrics	20
4.5 Results and Discussion	21
CHAPTER 5: CONCLUSION AND FUTURE SCOPE	**22**
5.1 Conclusion	22
5.2 Future Scope	22
References	23

LIST OF FIGURES

Figure 1. End-to-end diabetes prediction workflow.....17
Figure 2. Notebook training and Streamlit inference.....19

LIST OF TABLES

Table 1. Comparison of selected diabetes prediction studies....13
Table 2. Dataset size and class distribution....14
Table 3. Data preparation and preprocessing sequence....15
Table 4. Dataset and classifier settings....16
Table 5. Project modules and responsibilities....18
Table 6. Project files and purpose....19
Table 7. Recorded holdout performance of four models....20
Table 8. Performance metrics used in the evaluation....20

CHAPTER 1: INTRODUCTION

1.1 Introduction and Problem Statement

Diabetes is a chronic condition for which diagnosis and treatment require appropriate clinical assessment. Machine learning can demonstrate how labelled health records support binary classification, but a model output cannot establish disease. NIDDK explains that diabetes diagnosis relies on blood tests and that people should not diagnose themselves [1].

The commonly used Pima diabetes records include pregnancies, glucose, blood pressure, skin thickness, insulin, BMI, pedigree function, age, and a binary outcome [2], [6]. Pearson documents that zero values in some fields can disguise missing measurements [3]. These data-quality and population limits mean benchmark results do not establish performance in other groups or in current clinical care.

This project assembles an end-to-end educational workflow from CSV inspection through preprocessing, classifier comparison, and a Streamlit demonstration. It uses the supplied 768-row dataset, compares six scikit-learn models on one stratified test set, and exports a Random Forest model. The app accepts eight numeric inputs; its result is a dataset-derived classification and is not a diagnosis.

Problem statement: develop a classroom workflow that prepares the supplied diabetes data, compares common supervised classifiers on a held-out split, and demonstrates a model output in a small interface while describing data-leakage, preprocessing-consistency, and medical-interpretation limits.

1.2 Objectives of the Project

The primary objective is to implement and document an educational diabetes outcome classifier, compare six supervised models with consistent test-set metrics, and demonstrate the selected Random Forest through an interactive Streamlit application.

- Inspect the supplied CSV for its row count, eight predictors, binary outcome, class balance, and zero-coded measurements.
- Replace selected zero-coded measurements with column means and create a stratified 80:20 train-test split.
- Train and compare six classifiers using accuracy, precision, recall, F1-score, and ROC-AUC on one holdout set.
- Export the selected Random Forest and connect it to a Streamlit form that accepts all eight predictors.
- Document evaluation limits, preprocessing inconsistency, appropriate educational use, and steps for stronger validation.

1.3 Scope of the Project

The project covers local analysis of diabetes.csv, descriptive statistics and plots, handling selected zero-coded entries, one stratified holdout experiment, model comparison, and a local Streamlit interface. The target is Outcome=0 or 1. The interface expects pregnancies, glucose, blood pressure, skin thickness, insulin, BMI, DiabetesPedigreeFunction, and age, then displays its prediction and positive-class score. The saved estimator is Random Forest.

The supplied artifacts include the analysis notebook, diabetes.csv, a root model pickle, and App/app.py with a second model copy, README, and requirements. The notebook imputes selected zeros before training; the app sends raw form inputs to the estimator. This mismatch is a reproducibility limitation, not a validated deployment pipeline.

Out of scope are clinical diagnosis or treatment, prospective or external validation, calibrated risk estimates, patient-record integration, privacy/security certification, and regulatory approval. The data represent a historical benchmark schema rather than a current clinical cohort. The project folder does not include a source URL or license note, so the exact download copy and reuse terms cannot be confirmed. The intended users are students and project reviewers.

CHAPTER 2: LITERATURE SURVEY

2.1 Review of Existing Work

Research on diabetes classification compares statistical and machine-learning methods on benchmark datasets. The studies below provide context for this project design; their reported metrics use different preprocessing and evaluation setups and should not be compared numerically with the present holdout results.

Smith et al. tested the ADAP learning algorithm to forecast diabetes onset in a high-risk Pima population [2]. Their early work provides historical context for predictive modeling on these records, but its target and evaluation design differ from this project.

Pearson examined disguised missing data and discussed implausible zero values in Pima dataset fields [3]. This is relevant because the supplied notebook treats selected zeros as missing. The paper supports investigating data quality; it does not establish column-mean imputation as the best method.

Sisodia and Sisodia compared Naive Bayes, decision tree, and support vector machine methods for diabetes prediction [4]. Their work supports evaluating several classifiers instead of assuming one method is best. The published scores are not directly comparable with this project's split.

Chang et al. compared Naive Bayes, Random Forest, and J48 decision tree methods on the Pima data [5]. Their analysis reports complementary measures and feature-selection experiments. It reinforces that model rankings can depend on features and evaluation metrics.

The UCI Machine Learning Repository describes a Pima Indians Diabetes Database with the familiar eight-predictor schema [6]. The local CSV matches this schema, but the project folder does not retain a download URL or license, so exact provenance remains unverified.

Table 1. Comparison of selected diabetes prediction studies

Study	Data and task	Method or finding	Relevance and limit
Smith et al. [2]	Pima population; forecast diabetes onset.	Applied ADAP; evaluated sensitivity, specificity, and ROC.	Historical context; study design differs from this split.
Pearson [3]	Pima records and disguised missing values.	Documents implausible zeros in measured fields.	Supports data review; not evidence for mean imputation.
Sisodia and Sisodia [4]	Diabetes classification with benchmark data.	Compared Naive Bayes, decision tree, and SVM.	Supports multiple baselines; another evaluation setup.
Chang et al. [5]	Pima dataset with eight attributes.	Compared Naive Bayes, Random Forest, and J48.	Shows results vary with features and chosen metrics.
UCI record [6]	Pima Indians Diabetes Database schema.	Describes the common benchmark feature set.	Context only; local CSV provenance remains unverified.

2.2 Summary of Literature Survey

Prior work establishes the Pima records as a recurring benchmark for binary diabetes classification and highlights how model choice, feature selection, and data handling affect results [2]-[5]. Pearson's analysis makes zero-coded missingness a data-quality concern rather than a neutral value [3]. Reported literature scores describe their own experimental conditions; they are not a baseline for this project unless the dataset copy, preprocessing, split, and metrics align.

Classical classifiers remain useful baselines because they are practical to train and compare. This project uses six models on the same holdout partition and reports several metrics. Its contribution is an integrated classroom demonstration; evidence remains limited to the supplied CSV and one recorded split.

2.3 Research Gap

Prior studies compare diabetes classifiers and describe data-quality issues on the Pima benchmark [2]-[5]. The practical gap in this project is reproducibility between model evaluation and application inference: the notebook imputes selected zeros before fitting, while the app sends raw numeric form values to a serialized Random Forest. A shared, fitted preprocessing pipeline and stronger validation are needed to close this gap.

CHAPTER 3: DATASET AND PROPOSED METHODOLOGY

3.1 Dataset Description / Input Data

The supplied project folder contains diabetes.csv, the analysis notebook, serialized model files, and a Streamlit app. The CSV has 768 records, eight predictors, and a binary Outcome field. The schema matches the commonly circulated Pima diabetes feature set, including glucose, blood pressure, insulin, BMI, pedigree function, and age [2], [6].

The file contains 500 Outcome=0 records and 268 Outcome=1 records. Its columns are Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, Age, and Outcome. The local project does not record the download URL, version, license, or source note. The schema is consistent with a Pima-style benchmark, but this match is not proof of the exact CSV provenance or permission to redistribute it.

Outcome=0 represents 65.1% of records and Outcome=1 represents 34.9%. This imbalance makes accuracy alone insufficient; class-specific recall and other measures should also be reviewed. The dataset does not establish clinical representativeness.

Table 2. Dataset size and class distribution

Stage	Total records	Outcome 0	Outcome 1
Source CSV	768	500	268
After mean imputation	768	500	268
Training partition	614	400	214
Test partition	154	100	54

3.2 Data Preprocessing / Input Preparation

The notebook checks explicit nulls, then maps zero to missing in Glucose, BloodPressure, SkinThickness, Insulin, and BMI. Their zero counts are 5, 35, 227, 374, and 11. It fills these fields with their column means and confirms no remaining nulls. This is the recorded baseline; zero can be meaningful in some data contexts, so the rule requires domain review.

A stratified 80:20 split with random_state=42 creates 614 training and 154 test records. Scaling is fitted on training inputs and used for KNN, Logistic Regression, and SVM; tree models use unscaled values. In the notebook, however, imputation happens before the split, so test-set values contribute to the means. A leakage-safe pipeline should fit imputation only within training folds.

At inference time, App/app.py collects eight numeric values and orders them for the saved estimator. It does not map zeros to missing or apply the notebook's imputation means. The app input path therefore differs from the evaluated workflow; the transformation should be packaged with the model and used in both training and prediction.

Table 3. Data preparation and preprocessing sequence

Step	Operation	Purpose
1	Map zero to missing in five measurements.	Represent coded missing values explicitly.
2	Fill affected fields using column means.	Use the notebook's baseline imputation.
3	Compute means before train-test split.	Test rows contribute to these means.
4	Stratify Outcome; split 80:20, state 42.	Retain class proportions in both partitions.
5	Scale KNN, Logistic Regression, and SVM inputs.	Apply training-fitted scaling to test rows.
6	Fit six classifiers and score the holdout.	Compare accuracy, precision, recall, F1, ROC-AUC.

3.3 Algorithm / Model / Core Technique

The project is a supervised binary-classification experiment. KNN assigns a class from nearby records and uses standardized features; Logistic Regression models class probability with a linear log-odds function; and the RBF-kernel SVM learns a nonlinear boundary. These distance- or scale-sensitive models use standardized inputs as configured in the notebook.

The Decision Tree recursively partitions feature values. Random Forest combines 200 trees with random_state=42. Gaussian Naive Bayes estimates class likelihoods under a conditional-independence assumption. The notebook saves the Random Forest as the selected estimator. The app displays a positive-class score when available, but that score is not a calibrated medical risk.

All six classifiers use the same prepared training and test rows. The notebook compares accuracy, precision, recall, F1-score, and ROC-AUC; the project ranks the saved Random Forest highest by ROC-AUC. This single experiment does not establish that the model is optimal for another dataset or population.

Table 4. Dataset and classifier settings

Component	Setting	Project value
Dataset	Rows / predictors	768 / 8
Split	test_size / random_state	0.20 / 42
KNN	n_neighbors / input	5 / standardized
Logistic Regression	max_iter / input	1,000 / standardized
SVM	kernel / probability	RBF / True
Decision Tree	configuration	scikit-learn defaults
Random Forest	n_estimators / seed	200 / 42
Gaussian Naive Bayes	configuration	GaussianNB defaults
Model selection	ranking metric	ROC-AUC

3.4 Development, Testing and Validation

The notebook records data inspection, zero replacement, model training, metric calculation, and export of the selected estimator. The stratified split contains 614 training rows and 154 test rows; the test set has 100 Outcome=0 and 54 Outcome=1 records. All six classifiers use this same holdout partition.

Validation is limited to one random holdout evaluation. The supplied files do not record repeated cross-validation, hyperparameter search, threshold tuning, confidence intervals, or external validation. The saved model can be loaded by the app, but those artifacts do not provide evidence of clinical performance or model calibration.

The notebook saves the Random Forest estimator after comparing six classifiers on the held-out split. The reported results are copied from its recorded output and are not independently recomputed here. Training-time measurements, package versions, and hardware details are not recorded in the supplied project files.

3.5 Proposed Solution and System Workflow

The proposed solution is a data-analysis notebook plus an interactive Streamlit demonstration. The notebook prepares the CSV, trains and compares six classifiers, and saves a Random Forest estimator. The app collects eight numeric fields and displays a class and score. Its raw-input path does not repeat the notebook's zero-value treatment. The workflow proceeds through these steps:

1. The notebook loads diabetes.csv and checks its records, fields, and outcome classes.

2. The notebook maps selected zero-coded measurements to missing values and fills them with column means.
3. The prepared rows are split into stratified training and test partitions; selected models use scaled inputs.
4. Six classifiers are trained and compared using the same held-out test records.
5. The selected Random Forest is serialized and copied for use by the Streamlit interface.
6. A user enters eight values in the app, which displays the model output and an educational-use caution.

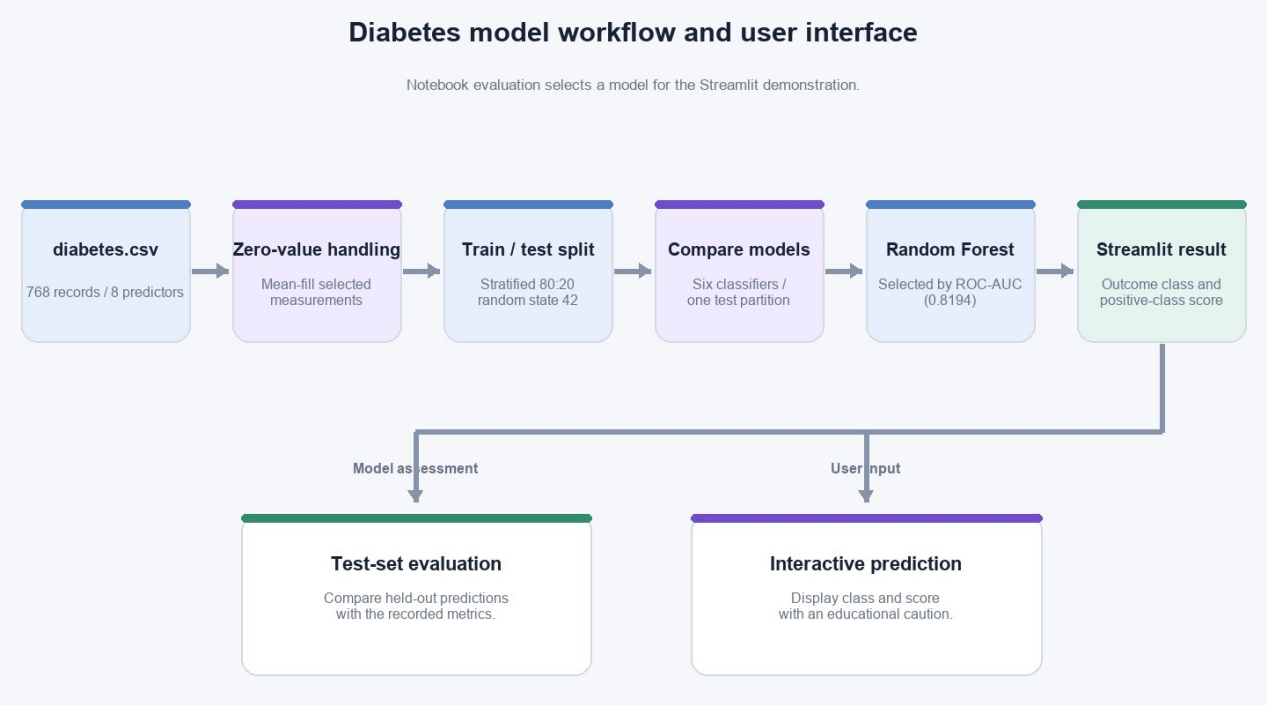


Figure 1. End-to-end diabetes prediction workflow. Notebook evaluation and app inference share the selected model.

CHAPTER 4: IMPLEMENTATION AND RESULTS

4.1 Development Environment and Tools

The project uses Python, Jupyter Notebook, pandas, NumPy, Matplotlib, Seaborn, scikit-learn, joblib, and Streamlit. The notebook trains a RandomForestClassifier with 200 estimators and random_state=42. The app loads the serialized model with joblib and builds an eight-feature input row. Exact Python, package, operating-system, and hardware versions are not recorded.

The project uses a local CSV and pickle-based workflow. App/app.py relies on Streamlit inputs and pandas to assemble model features; App/requirements.txt lists application dependencies and App/README.md documents local launch steps. The project does not include a pinned environment lock file, so exact dependency versions should not be inferred.

4.2 Implementation

The notebook contains exploratory checks, summary statistics, plots, zero-value treatment, a stratified split, scaling for selected models, six classifier fits, and evaluation metrics. It exports a Random Forest estimator. The Streamlit application collects Pregnancies, Glucose, BloodPressure, SkinThickness, Insulin, BMI, DiabetesPedigreeFunction, and Age, then displays the predicted Outcome and positive-class score when supported.

Table 5. Project modules and responsibilities

Module	Implementation	Responsibility
Data	diabetes.csv	Supply 768 records, eight predictors, and Outcome.
Analysis	Jupyter notebook	Inspect, prepare, plot, train, and evaluate.
Preprocessing	Notebook workflow	Map selected zeros and mean-impute fields.
Training	Six scikit-learn models	Fit and compare a common holdout experiment.
Model artifact	diabetes_model.pkl	Provide the saved Random Forest estimator.
User interface	App/app.py and Streamlit	Collect eight values and display prediction.
Run notes	App/README.md and requirements.txt	Document dependencies and local launch steps.

The application has three main parts: the Streamlit form, the serialized Random Forest, and the prediction display. It accepts eight numeric measurements and orders them for model inference, but the classifier artifact does not include the notebook’s

zero-value transformation or imputation. The app therefore sends raw form values to a model evaluated after mean imputation, so training and prediction inputs are not prepared in the same way. This should be corrected before interpreting app output as a faithful reproduction of notebook evaluation.

Table 6. Project files and purpose

Artifact	Key contents	Purpose
diabetes.csv	768 rows; eight predictors and Outcome.	Input data; source/license not recorded locally.
Analysis notebook	Plots, preprocessing, model comparison, metrics.	Records the experimental workflow.
App and model files	App/app.py; diabetes_model.pkl; run notes.	Educational Streamlit prediction prototype.

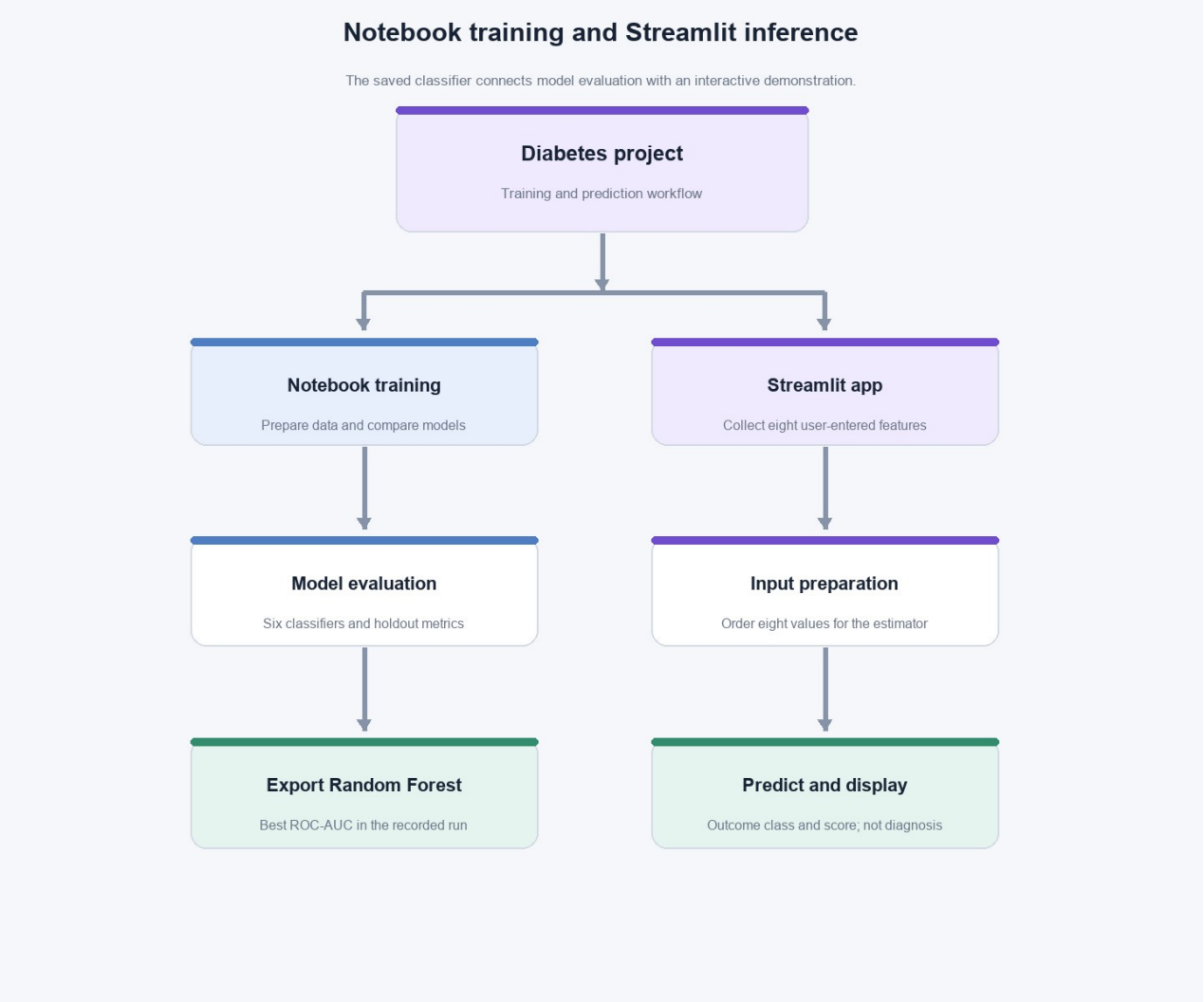


Figure 2. Notebook training and Streamlit inference workflow.

The Streamlit app loads the saved estimator and displays its predicted class and positive-class score. That score is a model output, not a calibrated individual risk. Because the app does not apply the notebook's zero handling, users can submit values that the model was not evaluated on in that form. The interface is an educational demonstration and must not be used for diagnosis or treatment decisions [1].

4.3 Experimental Results

The values below are copied from the notebook's recorded output for the 154-row test partition (100 Outcome=0 and 54 Outcome=1). Outcome=1 is treated as the positive class for precision, recall, and F1-score. Four models are shown here; the notebook also evaluated Gaussian Naive Bayes and Decision Tree.

Table 7. Recorded holdout performance of four models

Model	Accuracy	Precision	Recall	F1-score	ROC-AUC	AUC rank
Random Forest	0.7468	0.6596	0.5741	0.6139	0.8194	1
Logistic Regression	0.6948	0.5778	0.4815	0.5253	0.8117	2
KNN (k=5)	0.7338	0.6182	0.6296	0.6239	0.7975	3
SVM (RBF)	0.7338	0.6444	0.5370	0.5859	0.7912	4

Random Forest records the highest accuracy (0.7468) and ROC-AUC (0.8194) among the six models. KNN has the highest positive-class recall (0.6296) and F1-score (0.6239). Gaussian Naive Bayes recall is 0.6296; Decision Tree records the lowest ROC-AUC (0.6172).

4.4 Performance Evaluation Metrics

Accuracy is the fraction of correct predictions. Precision is the share of positive predictions that are correct; recall (sensitivity) is the share of actual positive records identified. F1-score is their harmonic mean. ROC-AUC measures ranking discrimination across thresholds. Here, Outcome=1 is the positive class. These metrics describe one held-out split, not clinical usefulness [7].

Table 8. Performance metrics used in the evaluation

Metric	Definition	Interpretation in this project
Accuracy	(TP + TN) / (TP + TN + FP + FN)	Share of 154 held-out predictions correct.
Precision	TP / (TP + FP)	Among Outcome=1 predictions, share truly Outcome=1.
Recall	TP / (TP + FN)	Among actual Outcome=1 rows, share identified.
F1-score	2 x (Precision x Recall) / (Precision + Recall)	Balances precision and recall for Outcome=1.
ROC-AUC	Area under the receiver operating characteristic curve.	Ranks positive records across classification thresholds.

For the Random Forest confusion matrix, TN=84, FP=16, FN=23, and TP=31, with actual classes as rows and predicted classes as columns. The test partition contains 54 positive records;

23 of those positive records were classified as Outcome=0. The result shows why class-specific errors matter alongside aggregate accuracy.

4.5 Results and Discussion

Random Forest has the highest ROC-AUC (0.8194) and accuracy (0.7468) in this comparison. KNN leads on Outcome=1 recall and F1-score; Gaussian Naive Bayes also reaches 0.6296 recall. The best model therefore depends on which metric is prioritized.

No training-time comparison is reported because the supplied notebook output does not provide a verified, stable timing record.

The results indicate moderate separation on one test split with 100 negative and 54 positive records. They are limited by the single holdout, mean imputation before splitting, absence of external validation and tuning, and inconsistent preprocessing in the app. Performance on the benchmark does not establish generalization to other populations or clinical practice.

The app demonstrates inference from eight form values and displays a model class and score. Its raw-input path differs from notebook preprocessing, and the score is not a validated risk estimate. The output must not be used to diagnose, treat, or rule out diabetes; appropriate blood tests and professional interpretation are required [1].

The project does not record cross-validation, calibration, an independent test cohort, pinned package versions, or a formal app-parity evaluation. The data download source and license are also absent from the supplied folder. These gaps should be addressed before the model or dataset is redistributed or considered for any use beyond coursework.

CHAPTER 5: CONCLUSION AND FUTURE SCOPE

5.1 Conclusion

This project implements an educational diabetes classification workflow using the supplied CSV, exploratory analysis, selected zero-value imputation, and six supervised classifiers on a stratified 80:20 holdout split. Random Forest records the highest ROC-AUC (0.8194) and accuracy (74.68%), while KNN has the highest Outcome=1 recall and F1-score. The results show why model comparison should consider several metrics rather than accuracy alone.

The Streamlit interface accepts eight feature values and displays the serialized Random Forest model's prediction and positive-class score. The project demonstrates an end-to-end classroom prototype, but the app does not reproduce the notebook's missing-value handling. Imputation is also performed before the split, so the recorded scores should be treated as an initial experiment rather than robust evidence of real-world performance.

The principal limitations are the single holdout, full-data imputation before splitting, inconsistent app preprocessing, and missing dataset provenance and license information. The model is not clinically validated, and its result is not a diagnosis. The project objectives are met as a prototype and model comparison; the following work is needed for reproducibility and stronger evidence.

5.2 Future Scope

- Build a scikit-learn pipeline that includes zero handling, imputation, and the classifier; fit preprocessing only within training folds and export the complete pipeline.
- Use stratified cross-validation and an independent test set; report uncertainty intervals and threshold-dependent confusion matrices.
- Analyze class-specific errors, calibration, and sensitivity/specificity trade-offs before choosing any operating threshold.
- Validate on independent, more recent, and demographically diverse data; assess population shift and fairness before making generalization claims.
- Add input validation, explicit units, missing-value handling, package versions, and a model card to the Streamlit application.
- Test that app inference uses the same fitted preprocessing pipeline as model training and evaluation.
- Confirm the exact dataset source, version, citation, and license before external redistribution or reuse.
- Complete the guide designation and required signatures, and retain the recorded split and metric outputs with the submission.

REFERENCES

[1] National Institute of Diabetes and Digestive and Kidney Diseases, “Diabetes Tests & Diagnosis.” [Online]. Available: https://www.niddk.nih.gov/health-information/diabetes/overview/tests-diagnosis. Accessed: Nov. 3, 2026.

[2] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, “Using the ADAP learning algorithm to forecast the onset of diabetes mellitus,” Proc. Annu. Symp. Comput. Appl. Med. Care, 1988, pp. 261–265. Available: https://pmc.ncbi.nlm.nih.gov/articles/PMC2245318/.

[3] R. K. Pearson, “The problem of disguised missing data,” SIGKDD Explorations, vol. 8, no. 1, pp. 83–92, 2006. doi: 10.1145/1147234.1147247. Available: https://www.kdd.org/exploration_files/12-Pearson.pdf.

[4] D. Sisodia and D. S. Sisodia, “Prediction of diabetes using classification algorithms,” Procedia Computer Science, vol. 132, pp. 1578–1585, 2018. doi: 10.1016/j.procs.2018.05.122.

[5] V. Chang, J. Bailey, Q. A. Xu, and Z. Sun, “Pima Indians diabetes mellitus classification based on machine learning algorithms,” Neural Computing and Applications, vol. 35, pp. 16157–16173, 2023. doi: 10.1007/s00521-022-07049-z.

[6] UCI Machine Learning Repository, “Pima Indians Diabetes Database.” [Online]. Available: https://archive.ics.uci.edu/dataset/34/pima+indians+diabetes. Accessed: Nov. 3, 2026. Cited for schema context; local CSV provenance is not confirmed.

[7] scikit-learn developers, “Classification metrics,” scikit-learn API documentation. Available: https://scikit-learn.org/stable/api/sklearn.metrics.html. Accessed: Nov. 3, 2026.

[8] Streamlit, “st.number_input,” Streamlit API reference. Available: https://docs.streamlit.io/develop/api-reference/widgets/st.number_input. Accessed: Nov. 3, 2026.

[9] Diabetes Prediction project artifacts, local project folder, 2026. Files consulted: diabetes.csv, Diabetes_Prediction_Notebook_with_Plots_2.ipynb, diabetes_model.pkl, App/app.py, App/requirements.txt, and App/README.md.

[10] Diabetes Prediction Notebook, local project artifact, 2026. Recorded split, model parameters, test metrics, and confusion matrix consulted for this report; values were transcribed and not independently recomputed.